

Concepts Authoring Guide

Use this guide when you need to describe **business concepts, important terms, identities, classifications, states, and relationships**.

This guide is about **what things mean** in the domain before you talk about processes, screens, or implementation.

What belongs here

Capture things like:

- important business entities
- people, roles, or external actors
- classifications or statuses
- identifiers
- lifecycle states
- relationships between concepts

What does not belong here

- a full process flow — capture that in the Story guide
- screen behavior — capture that in the Interactions guide
- detailed technical storage structures unless they matter to business meaning

Core things to capture

Describe:

- the **concept name**
- what it **means**
- how you **recognize** one
- what makes it **different** from similar concepts
- what key properties matter in business terms
- whether it has states, classifications, or examples

Core relationships to capture

Make these links explicit in your text:

- concept -> related concept
- owner/container -> contained thing
- concept -> actor that uses it
- concept -> story where it appears
- concept -> status/classification/state

Modus operandi

When eliciting concepts, work from language and meaning:

1. list the important nouns and named things in the domain
2. merge duplicates and separate look-alikes
3. define each concept in plain language
4. identify how one instance is recognized
5. capture important relationships
6. add states, examples, and distinctions only where they add clarity

Focus on business meaning, not on database or UI representation.

Prompt set

Use questions like these:

- what is this thing?
- how do we know one instance from another?
- what information about it matters to the business?
- what states can it be in?
- what other concepts does it relate to?
- what is often confused with it?
- where does it appear in the main stories or screens?

Free-text intake template

Use this template when gathering notes:

- **Concept name:**
- **Meaning:**
- **How to recognize one:**

- **Key business properties:**
- **Possible states or classifications:**
- **Related concepts:**
- **Examples:**
- **Where it appears in stories or interactions:**
- **What it is often confused with:**

Worked example

Example: leave request

- **Concept name:** Leave Request
- **Meaning:** a request by an employee to take time away from work for a specific period
- **How to recognize one:** it belongs to one employee and one requested date range
- **Key business properties:** requester, leave type, start date, end date, reason, decision
- **Possible states:** draft, submitted, approved, rejected, cancelled
- **Related concepts:** employee, leave balance, manager approval
- **Examples:** summer vacation request, medical leave request
- **Appears in:** leave request submission story, leave request form, manager review screen
- **Often confused with:** leave balance, approved leave calendar entry

How this becomes YAML

A technical modeler or AI agent can transform this into a concepts layer by:

- turning the named business concepts into concepts or actors
- recording identity, states, and examples where useful
- mapping relationships between concepts
- linking concepts to stories, rules, and interactions elsewhere in the Blueprint

You do not need to decide formal IDs or low-level structure.

Common mistakes

- naming a concept without defining what it means
- describing technical storage instead of business meaning
- missing identity or distinguishing characteristics

- forgetting relationships between concepts
- duplicating the same concept under slightly different names
- mixing concept definition with full process description

